Physics & AI (LLMs) for CFD Simulations

Improved PINNs for crystal growth · cryogenic H_2/N_2 systems · agentic simulation

QuaSi AI

October 6, 2026

Outline

The talk in one sentence

CFD is accurate but slow and expert-heavy. **Physics-informed ML** makes it fast, **LLM agents** make it accessible, and **validation** keeps both honest.

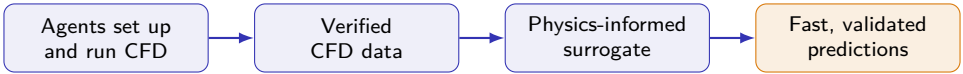
Why combine physics and AI for CFD?

The bottleneck

- High-fidelity multi-physics CFD needs fine meshes and long runs.
- The real cost is the **sweep**: every new rotation rate, heat flux or geometry is a new simulation.
- Setting up a valid case needs scarce solver expertise.

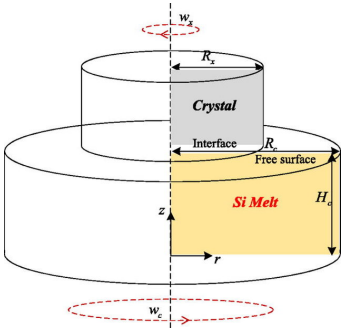
Two complementary levers

- **Physics-informed surrogates** (PINNs, Gaussian processes): learn the solution once, then query it in milliseconds; the PDE constrains training.
- **LLM agents** (Dyad, AutoFOAM): turn a plain-language request into a runnable, checked model.



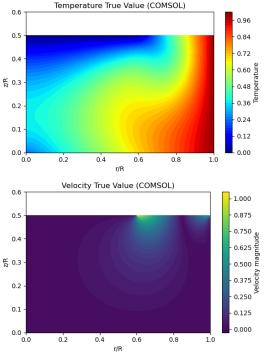
Key idea: Keep the governing physics, cut the cost per query, lower the set-up barrier, and always validate.

Use case: Czochralski (CZ) silicon crystal growth



Axisymmetric melt domain in (r, z)

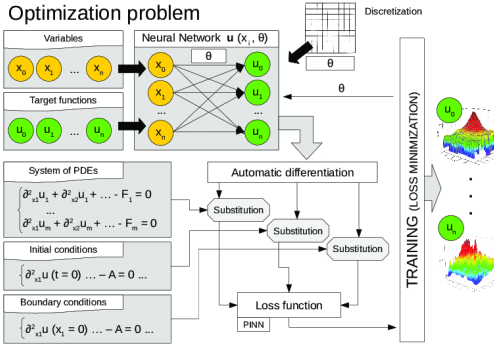
- A seed crystal is pulled from molten silicon; crystal (ω_s) and crucible (ω_c) rotate.
- Coupled physics: Navier–Stokes + energy, Boussinesq buoyancy, rotating walls ($u_\theta = \omega r$).
- Five unknown fields per point: u_r, u_z, u_θ, p, T.
- These thermal and flow fields largely determine crystal quality.



Reference T and $|\mathbf{u}|$ (normalised, COMSOL)

Key idea: Accurate CFD exists, but sweeping process settings (rotation, heater temperature) with it is slow.

Physics-informed neural networks (PINNs) in one slide



Network → automatic differentiation → PDE/BC/IC residuals → loss

1. A neural network $\hat{u}_\theta(x,t)$ represents the solution.
2. Automatic differentiation gives exact derivatives, so the PDE residual can be evaluated at any point.
3. One loss combines PDE, boundary (BC) and initial (IC) terms:

$$\mathcal{L} = \lambda_{\text{PDE}}\,\mathcal{L}_{\text{PDE}} + \lambda_{\text{BC/IC}}\,\mathcal{L}_{\text{BC/IC}}$$

The catch

The terms have very different gradient scales, so the weights λ decide what the network actually learns.

Key idea: Physics enters through the loss, so less labelled data is needed; but the loss balance must be handled with care.

Study design: two questions about trust

A. Adaptive PINN training

Question: if the loss weights adapt during training, is a lower PDE loss a better solution?

- **Baseline PINN**: fixed unit weights.
- **GNPINN**: normalises the PDE and BC/IC gradients.
- **AgenticPINN**: a rule-based controller re-weights the loss terms.

Tested on the 1D heat equation (exact solution known) and a simplified CZ thermal-fluid PINN.

B. CFD surrogate (Gaussian process)

Question: how well does a surrogate predict a *whole unseen* operating condition, field by field?

- 33 ANSYS Fluent reference cases: steady, laminar, 2D axisymmetric, 8181 nodes.
- Three one-parameter sweeps: heater-wall temperature T_{hot}, crucible rotation ω_c, crystal rotation ω_s.
- One complete case per sweep is held out for testing.

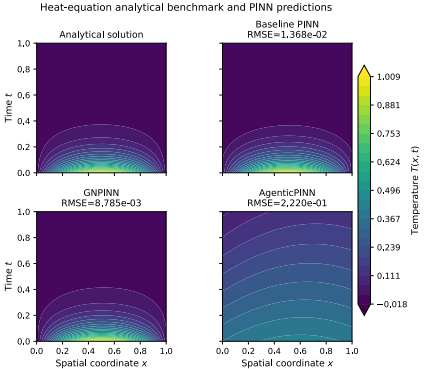
Key idea: Fair tests: in A only the training strategy changes; in B the test is a new process setting, not random points of known cases.

Result 1: lowest PDE residual $\neq$ best solution

Heat equation $\partial_t T = \partial_{xx} T$, exact solution $T = \sin(\pi x)\, e^{-\pi^2 t}$:

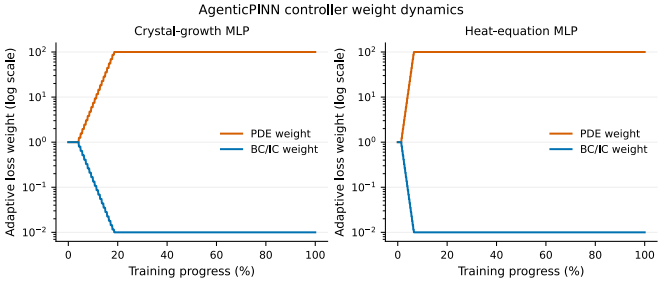
Trainer	PDE loss	BC/IC loss	Rel. L_2 error
Baseline	3.96×10^{-3}	1.15×10^{-3}	8.43×10^{-2}
GNPINN	9.74×10^{-3}	$\mathbf{9.05 \times 10^{-4}}$	$\mathbf{5.41 \times 10^{-2}}$
AgenticPINN	$\mathbf{8.22 \times 10^{-7}}$	2.00×10^{-1}	**1.368**

- AgenticPINN: PDE residual $\approx 4800\times$ smaller, BC/IC loss $\approx 173\times$ larger, and the field is wrong.
- Same pattern in the simplified CZ PINN: PDE residual $\approx 8\times$ smaller, BC/IC loss $\approx 790\times$ larger.



Key idea: Report PDE, BC and IC losses separately plus an external error; one aggregate loss can hide a failure.

Why? The controller silences the boundary conditions



AgenticPINN loss weights during training (log scale)

Mechanism

The controller balances gradient magnitudes and pushes $\lambda_{\mathrm{PDE}} \to 10^{2}$ and $\lambda_{\mathrm{BC/IC}} \to 10^{-2}$: a 10^{4} ratio within the first 20% of training.

Consequence

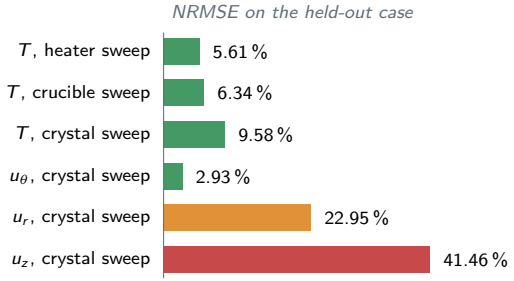
The network satisfies the PDE but no longer respects its boundary and initial conditions. Weighted total 2×10^{-3} looks excellent; the relative L_2 error is 1.37.

Key idea: Adaptive weighting is an optimisation tool, not an accuracy metric: always pair it with external validation.

Result 2: the surrogate's error depends on the field

Gaussian-process surrogate

- Learns $(r, z, s) \mapsto (u_r, u_z, u_\theta, p, T)$, where s is the swept parameter.
- Held out: $T_{\mathrm{hot}} = 1780\,\mathrm{K}$, $\omega_c = -4\,\mathrm{rpm}$, $\omega_s = 7\,\mathrm{rpm}$.
- NRMSE = RMSE / standard deviation of the field in the held-out case.
- Also gives a predictive uncertainty at every point.



Key idea: Temperature generalises well (5.6–9.6%); meridional velocities under crystal rotation do not (23–41%). Pooled scores hide this.

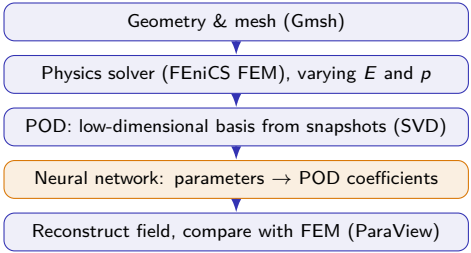
Cryogenic H_2/N_2 vessels: fast surrogates for design

Why it is hard

- Liquid H_2 ($\approx$20 K) and N_2 ($\approx$77 K): heat leak drives boil-off and pressure build-up.
- Vessel design couples thermal, fluid and structural loads; every design variant is a new simulation.

Approach (demonstrator)

- Open-source pipeline: a few solver runs, compressed with POD, then a neural network maps design parameters to the full field.
- Status: pressure-vessel demo on a coarse mesh; next are thermal loading (heat leak) and CFD coupling.



Key idea: Same recipe as for crystal growth: simulate a few cases, compress, learn the parameter-to-field map, check against the solver.

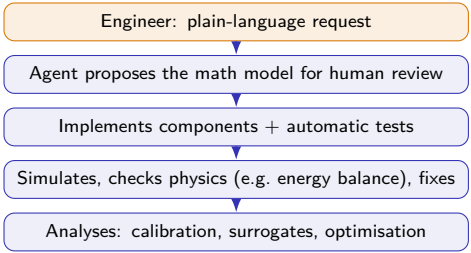
Dyad agent: an LLM agent for physical system models

What Dyad is

- A declarative modelling language from JuliaHub, built on Julia / ModelingToolkit.
- Combines physics-based modelling, scientific ML and agentic workflows.
- Acausal components with typed connectors (thermal, hydraulic, ...) enforce conservation at every connection.

Fit with this work

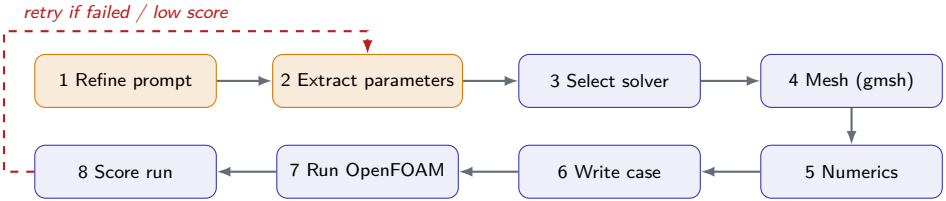
Candidate tool for system-level (0D/1D) cryogenic loops (tanks, lines, valves, heat leak), complementing field-level CFD.



Key idea: LLM agents are most trustworthy inside a constrained, checkable language, the same principle as in AutoFOAM.

AutoFOAM: from one sentence to a running OpenFOAM case

An OpenFOAM case is a dozen or more interlinked dictionaries; generic LLMs invent keywords and write inconsistent boundary conditions. AutoFOAM (fine-tuned 14B model, OpenFOAM v2412) splits the work:



LLM: language

Understands intent; schema-constrained JSON, so the output always parses.

Code: physics decisions

Rule-based routing to 7 approved solvers; 13 parameterised gmsh geometries.

Feedback: self-improving

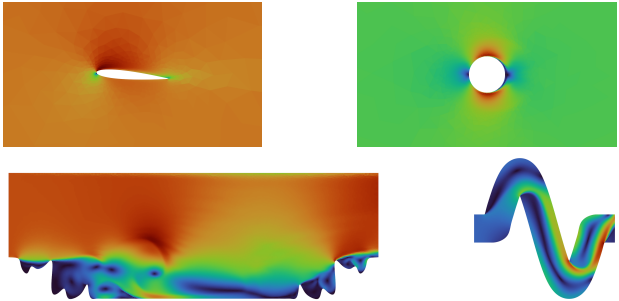
Runs are scored; failures are fixed and retried; good runs feed fine-tuning (SFT + DPO).

AutoFOAM results on 110 unseen prompts

Metric (out-of-distribution)	Value
Runs without `FOAM FATAL` error	**110/110**
Execution success	**100%**
Solver routing accuracy	**96.4%**
Geometry templates / solver families	13/13, 7/7
Mean run score (0–1)	0.64

Caveat

The score rewards convergence: a converged case can still be physically wrong, so a physics validator is the next step.



Velocity fields, each generated from a single prompt: airfoil, cylinder, periodic hill, S-bend

Demo: youtu.be/Kx17bfFIMSc
Code: github.com/AGN000/AutoFOAM

Key takeaways and next steps

1. **Objective $\neq$ accuracy:** a PINN can cut its PDE residual 4800$\times$ and still give the wrong field. Report PDE, BC and IC terms separately.
2. **Error is field-specific:** the CZ surrogate predicts temperature within 10% but axial velocity only within 41%.
3. **Agents remove set-up friction:** AutoFOAM runs 110/110 unseen prompts; Dyad builds checked system models.
4. **Validation is the common layer:** neither a low loss nor a converged run is proof on its own.

Next steps

- More PDEs and random seeds; higher-fidelity CZ models.
- Thermal (heat-leak) surrogates for cryogenic vessels.
- A physics validator inside the agent loop.

Key idea: Physics-informed surrogates + agentic automation + explicit validation $\Rightarrow$ fast, accessible and trustworthy CFD.

Acknowledgments

This talk presents the joint work of:

Niruta Chapagain, Rohit Raj	Adaptive PINN study for crystal growth (NeurIPS 2026 Sim2Science workshop paper)
Bertwin Kurisinkal Shine (TU Chemnitz)	CZ reference CFD: ANSYS Fluent set-up, parameter sweeps, mesh-independence study and certification
Arun Govind Neelan	AutoFOAM: agent design, fine-tuned model, training data and evaluation
A Seshaditya (QuaSi, Berlin)	Physics-informed ML project; AutoFOAM co-author

We also thank JuliaHub (Dyad) and the OpenFOAM, Gmsh and FEniCS communities. The CZ geometry and boundary conditions follow Huang et al. (2025).

Thank you! Questions?

quasi.digital github.com/adytiaa/quasi.ai

Selected references

- M. Raissi, P. Perdikaris, G. E. Karniadakis (2019). Physics-informed neural networks. *J. Comput. Phys.* 378, 686–707.
- S. Wang, Y. Teng, P. Perdikaris (2021). Understanding and mitigating gradient flow pathologies in PINNs. *SIAM J. Sci. Comput.* 43, A3055–A3081.
- W. Huang, S. Cao, R. Liu, D. Liu (2025). Thermal-fluid coupling in Czochralski silicon growth based on an improved PINN. *AIP Advances* 15, 105202. doi:10.1063/5.0271778.
- N. Chapagain et al. (2026). Component-level evaluation of adaptive PINN training for CFD-oriented crystal growth simulation. NeurIPS 2026 Sim2Science workshop.
- A. G. Neelan, A. Seshaditya. AutoFOAM: the self-refining autonomous OpenFOAM agent. github.com/AGN000/AutoFOAM.
- JuliaHub. Dyad documentation and AI agent. help.juliahub.com/dyad.